

बेसिक सिंटैक्स - इनपुट, आउटपुट, कमेंट्स और इंडेंटेशन।

Python एक सरल और आसानी से पढ़ी जाने वाली भाषा है। इसका सिंटैक्स स्वच्छ और पठनीय बनाने के लिए डिज़ाइन किया गया है।

① Python में कमेंट्स

कमेंट्स का उपयोग कोड को समझाने के लिए किया जाता है। Python सिंगल लाइन कमेंट्स के लिए “#” का उपयोग करता है।

यह एक सिंगल लाइन कमेंट है

इस लाइन को Python इंटरप्रेटर द्वारा अनदेखा कर दिया जाएगा

कमेंट्स कोड को समझने में आसान बनाते हैं।

② Python में इनपुट

यूज़र से इनपुट लेने के लिए हम `input()` फंक्शन का उपयोग करते हैं।

```
name = input("Enter your name:")  
age = input("Enter your age:")
```

`input()` हमेशा स्ट्रिंग के रूप में डेटा लौटाता है।

③ Python में आउटपुट

आउटपुट प्रदर्शित करने के लिए हम `print()` फंक्शन का उपयोग करते हैं।

```
print("Hello, World!")  
print("My name is", name)  
print("I am", age, "years old.")
```

`print()` टेक्स्ट, वेरिएबल्स या दोनों को प्रदर्शित कर सकता है।

④ Python में इंडेंटेशन

इंडेंटेशन का अर्थ है लाइन की शुरुआत में स्पेस देना।

Python कोड ब्लॉक्स को परिभाषित करने के लिए इंडेंटेशन का उपयोग करता है।

```
if age >= 18:  
    print("You are an adult.")  
else:  
    print("You are a minor.")
```

महत्वपूर्ण बिंदु :

- एक ब्लॉक में समान संख्या में स्पेस का उपयोग करें।
- आमतौर पर इंडेंटेशन के लिए 4 स्पेस का उपयोग किया जाता है।

सारांश :

#	→	कमेंट्स के लिए उपयोग किया जाता है।
input()	→	यूज़र से इनपुट लेने के लिए उपयोग किया जाता है।
print()	→	आउटपुट प्रदर्शित करने के लिए उपयोग किया जाता है।
Indentation	→	कोड ब्लॉक्स को परिभाषित करने के लिए उपयोग किया जाता है।

याद रखें : Python केस-सेंसिटिव है और इंडेंटेशन बहुत महत्वपूर्ण है। 😊

Python में बेसिक सिंटैक्स - इनपुट, आउटपुट का विवरण

Python एक सरल, आसानी से पढ़ी जाने वाली और उच्च स्तरीय प्रोग्रामिंग भाषा है। हर Python प्रोग्राम कुछ बेसिक सिंटैक्स नियमों का पालन करता है। किसी भी प्रोग्राम के दो सबसे महत्वपूर्ण भाग होते हैं - **Input** (डेटा लेना) और **Output** (परिणाम दिखाना)।

1. PYTHON का बेसिक सिंटैक्स

- Python केस-सेंसिटिव है (name, Name और NAME अलग-अलग माने जाते हैं)।
- स्टेटमेंट नई लाइन में लिखे जाते हैं।
- इंडेंटेशन (spaces) का उपयोग कोड के ब्लॉक्स को परिभाषित करने के लिए किया जाता है।
- कमेंट्स कोड को समझाने के लिए उपयोग किए जाते हैं।
- प्रोग्राम की एज़िक्यूशन पहली लाइन से शुरू होती है।

उदाहरण :

```
# यह मेरा पहला Python प्रोग्राम है
print("Hello, World!") # Output statement
print("Welcome to Python")
```

Output :

```
Hello, World!
Welcome to Python
```

2. PYTHON में इनपुट

रनटाइम पर यूजर से डेटा लेने के लिए `input()` फंक्शन का उपयोग किया जाता है। Python इस उद्देश्य के लिए `input()` फंक्शन प्रदान करता है।

- Syntax : `variable = input("message")`
- डिफॉल्ट रूप से `input()` डेटा को स्ट्रिंग के रूप में लेता है। आवश्यकता होने पर हम इसे `int`, `float` आदि में कन्वर्ट कर सकते हैं।

Example 1 : स्ट्रिंग इनपुट लेना

```
name = input("Enter your name : ")
print("Hello, ", name)
```

Sample Run :

```
Enter your name : Rahul
Hello, Rahul
```

Example 2 : इंटीजर इनपुट लेना

```
age = int(input("Enter your age : "))
print("You are", age, "years old.")
```

Sample Run :

```
Enter your age : 20
You are 20 years old.
```

Example 3 : एकाधिक इनपुट लेना

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))
sum = a + b
print("Sum = ", sum)
```

Sample Run :

```
Enter first number : 12
Enter second number : 25
Sum = 37
```

3. PYTHON में आउटपुट

स्क्रीन पर डेटा दिखाने के लिए `print()` फंक्शन का उपयोग किया जाता है।

- Syntax : `print(value 1, value 2, ..., sep = ' ', end = '\n')`
- `sep` : values के बीच का separator (डिफॉल्ट space होता है)
- `end` : अंत का कैरेक्टर (डिफॉल्ट नई लाइन होता है)

Example 1 : बेसिक आउटपुट

```
print("Python is easy to learn.")
```

Output :

```
Python is easy to learn.
```

Example 2 : अनेक मानों का आउटपुट

```
name = "Rahul"
marks = 95
print("Name :", name, "Marks :", marks)
```

Output :

```
Name : Rahul Marks : 95
```

Example 3 : `sep` और `end` का उपयोग

```
print("A", end = " - ")
print("B", end = " - ")
print("C") # default end is newline
```

Output :

```
A - B - C
```

- ★ सारांश :
- `input()` का उपयोग यूजर से इनपुट लेने के लिए किया जाता है।
 - `print()` का उपयोग आउटपुट दिखाने के लिए किया जाता है।
 - `input()` हमेशा स्ट्रिंग रिटर्न करता है, इसलिए `int()`, `float()` में कन्वर्ट करें यदि आवश्यक हो।
 - सही सिंटैक्स और इंडेंटेशन कोड को सही और आसानी से पढ़ने योग्य बनाते हैं।



मूल वाक्य रचना - Python में Comments और Indentation

===== . =====

Comments और indentation, Python में बहुत महत्वपूर्ण हैं। ये code को समझने में और program की संरचना (structure) को परिभाषित करने में मदद करते हैं।

1. Python में Comments

Comments का उपयोग code को समझाने के लिए किया जाता है। Program को execute करते समय Python, comments को अनदेखा (ignore) कर देता है।

Comments के प्रकार:

(A) Single Line Comment (एकल पंक्ति टिप्पणी)

यह # चिन्ह से शुरू होता है। उसी पंक्ति में # के बाद की सारी चीजों को comment माना जाता है।

उदाहरण (Example):

```
# This is a single line comment
x = 10      # x stores the value 10
y = 20      # y stores the value 20
print(x + y) # prints the sum
```

Output :

30

Comments execute नहीं होते। ये केवल इंसानों (humans) के लिए code समझने हेतु होते हैं।

(B) Multi Line Comment (बहु-पंक्ति टिप्पणी)

Python में multi-line comments के लिए कोई विशेष syntax नहीं है। इसके लिए हम triple quotes (`'''` या `"""`) का उपयोग कर सकते हैं।

उदाहरण (Example):

```
"""
This is a
multi line comment.
It can span across
multiple lines.
"""
a = 5
b = 15
print(a * b)
```

Output :

75

नोट (Note):

- Code क्या करता है, यह बताने के लिए comments का उपयोग करें।
- Comments, program को पढ़ने और maintain करने में आसान बनाते हैं।
- अच्छी आदत (Good practice): meaningful comments लिखें।

2. Python में Indentation

Indentation का अर्थ है किसी पंक्ति (line) की शुरुआत में spaces देना। Python, code के blocks को परिभाषित करने के लिए indentation का उपयोग करता है। यह अनिवार्य (mandatory) है।

उदाहरण 1: सही Indentation

```
if x > 10:
    print("x is greater than 10")
else:
    print("x is 10 or less")
```

Output (if x = 15) :

x is greater than 10

व्याख्या (Explanation):

- दोनों print() statements, if और else के अंतर्गत indented हैं।
- वे अपने-अपने (respective) blocks का हिस्सा हैं।

उदाहरण 2: गलत Indentation (IndentationError देगा)

```
if x > 10:  
print("x is greater than 10")  
else:  
    print("x is 10 or less")
```

Error :

```
IndentationError:  
expected an indented block
```

व्याख्या (Explanation):

- if के अंतर्गत print() indented नहीं है।
- Python संरचना (structure) को नहीं समझ पाता।

Indentation के बारे में महत्वपूर्ण बिंदु (Important Points):

- पूरे program में एक जैसी (consistent) indentation का उपयोग करें।
- Python प्रत्येक indentation level के लिए 4 spaces की सलाह देता है।
- Tabs नहीं, spaces का उपयोग करें (मिलाने से errors आ सकती हैं)।
- Indentation, statements के बीच के संबंध (relationship) को दर्शाता है।
- यह Python को यह जानने में मदद करता है कि कौन सा statement किस block का हिस्सा है।

उदाहरण: Nested Indentation

```
num = 8  
if num > 0:
```

Output :

```
Positive number  
Even number
```

```
print("Positive number")
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

★ अच्छे *comments* code को समझाते हैं। सही *indentation* code को काम करने योग्य बनाती है! 😊

© Vinay Soni

Python में Errors के प्रकार

जब हम कोई program लिखते हैं, तो errors आ सकती हैं। Errors वे समस्याएँ (problems) हैं जो program के सामान्य execution को रोक देती हैं।

Python की errors मुख्य रूप से दो प्रकार की होती हैं:

1. Syntax Errors

(Python interpreter द्वारा detect की जाती हैं)

2. Exceptions

(Program के execution के दौरान detect की जाती हैं)

1. Syntax Errors :

- ये errors तब होती हैं जब हम Python syntax के नियमों (rules) का उल्लंघन करते हैं।
- ये errors Python interpreter द्वारा compile time के दौरान detect की जाती हैं।
- अगर syntax error है, तो program run नहीं होगा।

उदाहरण (Example):

Code	Error	Reason (कारण)
<pre>print("Hello World"</pre>	<code>SyntaxError: invalid syntax</code>	Closing bracket) गायब है (missing)
<pre>x = 10 if x > 5 print(x)</pre>	<code>SyntaxError: expected ':'</code>	if condition के बाद colon (:) गायब है
<pre>print "Hello"</pre>	<code>SyntaxError: Missing parentheses in call to 'print'</code>	Python 3 में, print() एक function है।

2. Exceptions (Runtime Errors):

- ये errors तब होती हैं जब program चल (run) रहा होता है।
- इन्हें Runtime Errors भी कहा जाता है।
- Program सफलतापूर्वक compile हो जाता है, लेकिन exception के कारण execution के दौरान रुक जाता है।

Exceptions के महत्वपूर्ण प्रकार (Important Types):

S.No.	Exception	Description (विवरण)	Example Code	Error Message
1.	ZeroDivisionError	तब आता है जब हम किसी number को zero से divide करने की कोशिश करते हैं।	<pre>a = 10 b = 0 c = a / b</pre>	ZeroDivisionError: division by zero
2.	NameError	तब आता है जब कोई variable name नहीं मिलता (not found)।	<pre>print(x)</pre>	NameError: name 'x' is not defined
3.	TypeError	तब आता है जब कोई operation गलत data type पर लागू (apply) किया जाता है।	<pre>a = 10 print(a + "5")</pre>	TypeError: unsupported operand type(s) for +: 'int' and 'str'
4.	ValueError	तब आता है जब किसी function को सही type का, पर अमान्य (invalid) value का argument मिलता है।	<pre>int("abc")</pre>	ValueError: invalid literal for int() with base 10: 'abc'
5.	IndexError	तब आता है जब किसी sequence का index range से बाहर (out of range) होता है।	<pre>lst = [1, 2, 3] print(lst[5])</pre>	IndexError: list index out of range
6.	KeyError	तब आता है जब dictionary की कोई key नहीं मिलती (not found)।	<pre>d = {'a':1, 'b':2} print(d['c'])</pre>	KeyError: 'c'

7.	FileNotFoundError	तब आता है जब कोई file नहीं मिलती (not found)।	<code>f = open("data.txt")</code>	FileNotFoundError: [Errno 2] No such file or directory: 'data.txt'
----	--------------------------	---	-----------------------------------	--

नोट (Note):

- ★ Syntax Errors, program के run होने से पहले दिखाई देती हैं।
- ★ Exceptions, program के execution के दौरान आती हैं।

Program के crash होने से बचाने के लिए हमेशा try-except block का उपयोग करके exceptions को handle करें।

Python में Variables

==== . ==== . =====

Variable एक ऐसा नाम है जो memory के किसी location को दिया जाता है, जहाँ data store किया जाता है। यह हमें program में data को store, retrieve और manipulate करने की सुविधा देता है। Python एक dynamically typed language है, जिसका अर्थ है कि हमें variable का type स्पष्ट रूप से (explicitly) declare करने की आवश्यकता नहीं होती।

1. Variables Declare करना

- Python में, जैसे ही आप किसी variable को value assign करते हैं, वह बन (create हो) जाता है।
- Data type बताने की आवश्यकता नहीं है।
- उदाहरण (Example):

```
x = 10          # integer
name = "Rahul"  # string
price = 99.50    # float
is_pass = True   # boolean
```

2. Variable Names के नियम

- किसी letter (a-z, A-Z) या underscore (_) से शुरू होना चाहिए।
- letters, digits (0-9) और underscore (_) शामिल हो सकते हैं।
- किसी digit से शुरू नहीं हो सकता।
- !, @, #, \$, % जैसे special symbols का उपयोग नहीं कर सकते।
- Variable names case sensitive होते हैं।
- Python keywords को variable names के रूप में उपयोग नहीं कर सकते।

Valid Variable Names	Invalid Variable Names
age	1age (digit से शुरू होता है)
Age_1	total-marks (- शामिल है)
_value	%value (special symbol शामिल है)
totalMarks	for (keyword है)
x123	

3. Variables के प्रकार (Built-in Data Types)

Type	Description	Example
int	Integer (पूर्ण संख्याएँ)	a = 25
float	Decimal या real संख्याएँ	b = 12.75

4. Multiple Assignment

हम एक ही line में कई variables को values assign कर सकते हैं।

```
a, b, c = 1, 2, 3
x = y = z = 0
```

str	String (characters का समूह)	c = "Hello"
bool	Boolean (True या False)	d = True
list	क्रमबद्ध collection (mutable)	e = [1, 2, 3]
tuple	क्रमबद्ध collection (immutable)	f = (1, 2, 3)
dict	Key-value जोड़े (pairs)	g = { "id":1, "name": "Amit" }

5. Reassignment

- हम किसी भी समय variable की value बदल सकते हैं।
- उदाहरण (Example):

```
x = 10
print(x)      # 10
x = 25
print(x)      #
value changed -
25
```

6. Dynamic Typing

Python में, variable का type उसे assign की गई value से निर्धारित (determine) होता है, और यह dynamically बदल सकता है।

```
x = 5          # x एक integer
               है
x = "Python"   # अब x एक
               string है
x = 3.14       # अब x एक
               float है
```

हमें type declare करने की आवश्यकता नहीं है। Python इसे स्वचालित रूप से (automatically) handle करता है।

7. Variables का उपयोग

(i) Expressions में उपयोग

```
a = 10
b = 20
sum = a + b
print("Sum = ",
sum)  # Sum = 30
```

(ii) Input लेना और Variables में Store करना

```
name =
input("Enter your
name: ")
age =
int(input("Enter
your age: "))
print("Name: ",
name)
print("Age: ",
age)
```

नोट (Note):
Data को meaningful names देकर, variables किसी program को पढ़ने, modify करने और maintain करने में आसान बनाते हैं।
😊

★ **सारांश (Summary):** Python में एक variable, data को रखने के लिए एक नामित (named) storage location होता है। यह value assign करके बनाया जाता है और इसके लिए explicit declaration की आवश्यकता नहीं होती।

Python में मूलभूत डेटा प्रकार

Python में कई पूर्व-निर्मित (built-in) डेटा प्रकार होते हैं। सबसे अधिक उपयोग किए जाने वाले मूलभूत डेटा प्रकार हैं : संख्यात्मक प्रकार , स्ट्रिंग प्रकार , बूलियन प्रकार आदि।

1. संख्यात्मक प्रकार (NUMERIC TYPES)

संख्यात्मक प्रकार का उपयोग संख्यात्मक मानों को संग्रहीत करने के लिए किया जाता है।

प्रकार	विवरण	उदाहरण	कोड	आउटपुट
int	धनात्मक या ऋणात्मक पूर्ण संख्याएँ (दशमलव के बिना)	10, -25, 0	a = 10 type(a)	10 <class 'int'>
float	वास्तविक संख्याएँ (दशमलव मान)	3.14, -0.75, 2.0	b = 3.14 type(b)	3.14 <class 'float'>
complex	जटिल संख्याएँ (a + bj)	3+4j, -2j	c = 3+4j type(c)	(3+4j) <class 'complex'>

उदाहरण :

```
x = 10          # integer
y = -7          # integer
z = 2.56        # float
w = -3.5        # float
p = 5 + 2j      # complex
```

नोट :

- int में दशमलव भाग नहीं होता।
- float में दशमलव भाग होता है।
- complex संख्या a + bj के रूप में लिखी जाती है, जहाँ a और b वास्तविक संख्याएँ होती हैं तथा j काल्पनिक (imaginary) भाग होता है।

2. स्ट्रिंग प्रकार (STRING TYPE)

स्ट्रिंग वर्णों का अनुक्रम होता है, जो एकल उद्धरण (''), दोहरे उद्धरण (") या तिहरे उद्धरण (''' ''' या "" ""') में होता है।

उदाहरण :

```
s1 = 'Hello'
s2 = "Python Programming"
s3 = '''This is
      a multi-line
      string'''
```

स्ट्रिंग पर क्रियाएँ :

- जोड़ना (Concatenation) : 'abc' + 'def' → 'abcdef'
- दोहराव (Repetition) : 'ha' * 3 → 'hahaha'
- इंडेक्सिंग (Indexing) : s = 'Python', s[0] → 'P'
- स्लाइसिंग (Slicing) : s[1:4] → 'yth'
- लंबाई (Length) : len(s) → 6

स्ट्रिंग अपरिवर्तनीय (Immutable) है

एक बार बनने के बाद, हम स्ट्रिंग के व्यक्तिगत वर्णों को बदल नहीं सकते।

उदाहरण :

```
s = 'Hello'
s[0] = 'h' # Error
```

3. बूलियन प्रकार (BOOLEAN TYPE)

बूलियन प्रकार में केवल दो मान होते हैं : True या False. इसका उपयोग मुख्यतः शर्तों तथा तार्किक (logical) क्रियाओं में किया जाता है।

उदाहरण :

```
a = True
b = False
type(a) # <class 'bool'>
type(b) # <class 'bool'>
```

बूलियन क्रियाएँ :

- and : True और False → False
- or : True या False → True
- not : not True → False
not False → True

तुलना परिणाम (Boolean)

```
5 > 3 → True
5 == 3 → False
7 <= 7 → True
```

★ सारांश :
→ संख्यात्मक प्रकार : int, float, complex
→ स्ट्रिंग प्रकार : str
→ बूलियन प्रकार : bool (True / False)

ये मूलभूत डेटा प्रकार किसी भी Python प्रोग्राम की नींव होते हैं।

